

Futuruna: The Seven Symbols That Creates Clear Programming Without Compromise

Andreas Rudolph
Independent researcher, Copenhagen

Abstract

You open a file in an unfamiliar language. Within seconds, before you understand a single line, something in you either relaxes or tightens. That feeling has a number. We measured it for every major programming language and found that the syntax of every one of them answers at most two questions before you read a single word. You passively know how deep you are in nested blocks, and perhaps whether you are in a type or an expression. Everything else requires decoding: you read the keyword `fn` to learn it is a function, `struct` to learn it is a type.

A single innovation opens a third question. Place a dedicated character at the very start of each line, one that declares what kind of statement follows. We call this character a *rune*. The resulting language, Futuruna, has seven: `#` what exists, `>` what happens, `|` what must be true, `=` what is, `~` what flows, `@` where proofs stop, `?` prove it. Now the syntax passively tells you the statement kind, the type flow, and the block depth. Three questions answered for free.

We formalize this using three information-theoretic quantities: *optionality* (how many continuations exist), *clarity* (how distinguishable positions are), and *integration* (how many independent questions the syntax answers at once). Every existing language scores at most two on integration. Futuruna scores three. It compiles to native Rust with inferred ownership, unifies logic programming with pattern matching with legal default logic under one character, and provides a Z3 verification pipeline where each rune maps directly to a category in the formal methods backend.

Keywords: programming language design, integrated information theory, causal entropic forces, statement runes, formal verification, cognitive dimensionality

1 Imagine

Imagine a program where types, functions, rules, values, streams, effects, and verification are each announced by a single character at the start of the line:

```
# Condition = Sunny | Cloudy | Stormy
# Weather(day: String, temp: Float, condition: Condition)

> describe(w: Weather) → String {
  match w.condition {
    | Sunny → show(w.temp) + " C, sunny"
    | Stormy → show(w.temp) + " C, storm"
    | Cloudy → show(w.temp) + " C, cloudy"
  }
}

| advisory(w) → "all clear"
| advisory(w) → "heat warning" under w.temp > 35.0
| exception storm advisory(w) → "danger" under w.condition == Stormy
```

```

= today = Weather("today", 22.0, Sunny)
= alert = advisory(today)

@ print(today.day + ": " + describe(today) + " -- " + alert)

~ forecast = from_list([today, Weather("tomorrow", 40.0, Sunny),
    Weather("in 2 days", 18.0, Cloudy), Weather("in 3 days", 10.0, Stormy)])
    |> s_filter(|w: Weather| advisory(w) != "all clear")

= warning_count = s_count(forecast)
| has_warnings: warning_count → warning_count > 0

? has_warnings: n → {
    @ print("Upcoming warnings (" + show(n) + "):")
    ~ forecast | w → {
        @ print("  " + w.day + ": " + advisory(w) + " → " + describe(w))
    }
} else {
    @ print("No warnings -- all clear ahead")
}

```

The output:

```

today: 22.0 C, sunny -- all clear
Upcoming warnings (2):
  tomorrow: heat warning -> 40.0 C, sunny
  in 3 days: danger -> 10.0 C, storm

```

Seven runes, and they feed into each other. `#` defines types that every other rune uses. `>` defines `describe`, a function with `|` match arms inside. Three `|` lines define `advisory` as rules with override logic: the default is “all clear,” high temperature overrides to “heat warning,” storms override everything—no `if/else` chain, no Prolog. Notice the output: four days go in, but only two warnings come out. Day two (18°C, cloudy) triggers no override rule, so the default “all clear” holds and the filter silently removes it. The logic rules did the work; the stream carried the result. `=` binds `today` and calls the rules to get `alert`. `@` prints by combining the function and rule results. `~` declares a reactive stream that pipes `today` (an `=` binding) through a filter that calls the `|` rules—showing values flowing between runes. Then `|` appears again as a named invariant, and `?` interrogates it: `? has_warnings: n` captures the checked value, runs the pass block when it holds, and falls through to `else` when it does not.

Count the `|` character: it appears as match arms, as logic rules with exceptions, and as a named invariant that `?` interrogates. Three roles, one symbol, because they are all the same operation: *case analysis*.

A *rune* is a single special character placed at the very start of a line to declare what kind of statement follows. Futuruna has seven of them. Each one answers the same question: *what am I about to tell you?*

```

# what exists    > what happens    | what must be true    = what is
                ~ what flows      @ where proofs stop      ? prove it

```

In every existing language, you read the first *word* of a line to know what kind of statement it is: `fn`, `let`, `struct`, `if`, `println!`. In Futuruna, you read the first *character*. In the listing above, your eye scans `|`, `#`, `>`, `=`, `~`, `@`, `?` and knows what each line does before processing a single identifier.

That difference has a number. We call it d_{eff} : how many independent questions the syntax answers for free. In a language with $d_{\text{eff}} = 1$, you passively know one thing: how deep you are in nested blocks. Everything else requires reading. In a language with $d_{\text{eff}} = 3$, you passively know three things: what *kind* of statement you are on, what *types* flow through it, and where it *sits* in the structure. Three facts, delivered through three independent visual cues, before your brain processes a single word.

Every existing language scores $d_{\text{eff}} \leq 2$. Futuruna scores $d_{\text{eff}} = 3$.

2 What One Rune Replaces

Before the theory, look at what collapses.

Table 1: Each rune absorbs mechanisms scattered across multiple keywords and languages.

Rune	Unifies	Scattered across
#	Types + Effects + Traits	<code>class</code> , <code>struct</code> , <code>enum</code> , <code>interface</code> , <code>trait</code> , <code>type</code>
>	Functions + Actors + Modules	<code>fn</code> , <code>def</code> , <code>func</code> , <code>mod</code> , <code>actor</code>
	Logic + Matching + Effects + Law	Prolog (<code>:-</code>), Rust (<code>match</code>), Koka (<code>handle</code>), Catala (<code>rule</code>)
=	Binding + Monadic bind	<code>let</code> , <code>val</code> , <code>var</code> , Haskell <code><-</code>
~	Reactive streams	RxJS, Reactor, Akka Streams (libraries, not language)
@	IO + Imports + Build + FFI	<code>println!</code> , <code>use</code> , <code>import</code> , <code>build.gradle</code> , FFI
?	Tests + Asserts + Formal proofs	<code>assert!</code> , <code>#[test]</code> , external Z3 toolchain

The | row is the headline. Today, if you want logic programming, you learn Prolog. If you want legal rules with exceptions that override defaults, you learn Catala. If you want algebraic effects (a way to declare side effects like IO and state as values the caller can intercept and handle), you learn Koka. If you want pattern matching, you learn Rust or Haskell. Four different languages for four instances of the same idea: *case analysis under different regimes*.

In Futuruna, they are one rune.

```

-- Prolog-style inference
| taxable(person) → resident(person), has_income(person)

-- Catala-style default logic with exceptions
| tax_rate(person) → 0.20
| tax_rate(person) → 0.40 under person.income > 50000
| exception tax_rate(person) → 0.00 under person.income < 12000

-- Koka-style algebraic effect handler
| handle Console {
  | say(msg) → { @ print(msg); resume(()) }
  | ask(prompt) → resume("default")
} in greet("World")

-- ML-style pattern matching
match shape {
  | Circle(r) → 3.14 * r * r
  | Rectangle(w, h) → w * h
}

```

Four capabilities. One character. The reason they unify is structural: they are all case analysis. Logic clauses analyze cases over a search space. Pattern matching analyzes cases of a

value. Effect handlers analyze cases of a side effect. Legal rules analyze cases of a regulation. The regimes differ. The operation does not.

2.1 The ownership problem, dissolved

Here is a function in Rust:

```
fn process(data: &[String], prefix: &str) → Vec<String> {
    data.iter()
        .filter(|s| s.starts_with(prefix))
        .map(|s| s.clone())
        .collect()
}
```

The programmer had to decide: `&[String]` or `Vec<String>`? `&str` or `String`? `s.clone()` or `s.to_string()`? Every one of those decisions is an ownership annotation that Rust requires and the programmer provides.

Here is the same function in Futuruna:

```
> process(data: List(String), prefix: String) → List(String) {
    filter(data, |s| starts_with(s, prefix))
}
```

No `&`. No lifetime. No `.clone()`. The compiler sees that `data` is read-only and emits `&[String]` automatically. It sees that `prefix` is only borrowed and emits `&str`. It sees the filter closure captures by reference and skips the clone. The generated Rust is identical to—or better than—what a careful programmer would write.

In an adversarial comparison of 12 ownership patterns that commonly trip Rust newcomers, Futuruna produces valid Rust for all 12. It wins on 5 (zero clones where Rust requires manual annotation), ties on 7, loses on none.

2.2 Verification as syntax

The `?` rune scales from runtime check to formal proof without changing the source code:

```
| conservation: supply → supply >= 0 && supply <= max_supply
? conservation
```

In the interpreter, `? conservation` evaluates the predicate with current values. In compiled mode, it emits `debug_assert!()`. With `runa verify`, it translates the invariant into a format for automated theorem provers and invokes Z3 [8] (Microsoft’s open-source theorem prover) to prove the invariant holds for *all* inputs. One character. Three levels of assurance. The code never changes.

3 The Measurement

The claims above are not aesthetic opinions. One character changes how you navigate code. Futuruna occupies a region of design space no existing language touches. Both claims are the result of applying two theories from outside computer science to the structure of programming language syntax.

3.1 What we measured

Take any programming language. Collect a representative corpus of code. Classify every token into one of 22 categories—keyword, identifier, type name, operator, brace, parenthesis, and so

on. Count how often each category follows each other. Normalize to probabilities. The result is a 22×22 transition matrix: a complete description of which tokens tend to follow which, and with what probability.

This matrix is a small object—484 numbers. It turns out to encode an enormous amount about how a language feels to use. We compute three quantities from it. Each has a formal symbol and a human name.

Optionality (S_τ). From any position in the code, how many meaningfully different continuations exist? Start at a token category and follow the transition probabilities for three steps. If you fan out to many destinations, the syntax has high optionality—many things could come next. If you funnel into the same few endpoints, optionality is low—the syntax has already decided for you. Shannon entropy [1] measures this spread precisely. We write it S_τ and call it **optionality**: how open is your syntactic future?

Clarity (JSD). Optionality alone produces mush. A language where every token can follow every other has maximum optionality and zero readability—every position feels the same. **Clarity** measures how *distinguishable* different positions are. Can you tell whether you are inside a type declaration or a function body just from the local token context? Jensen-Shannon divergence (JSD) captures this: the average distance between the transition distributions of different token categories. High clarity means each position in the code has a distinctive feel. Low clarity means everything blurs together.

Integration (Φ). This is the one that matters most. Integration measures how many *questions the syntax answers before you read a single word*.

When you scan a line of code, your mind tracks several things at once: what kind of statement is this? What types flow through it? How deep am I in the block structure? Each of those is a question. In most languages, the syntax answers only one of them passively—block depth, conveyed by indentation or brace nesting. The rest require reading. You decode the keyword `fn` to learn it is a function. You decode the type annotation to learn what types flow. Each decoded answer costs time and attention.

In a language with high integration, the syntax answers multiple questions through *independent* visual cues—cues that carry non-redundant information. Knowing the first character tells you the statement kind; that tells you nothing about the types, which tells you nothing about the block depth. Three questions, three answers, arriving on three separate channels. Your brain processes them in parallel, the way a pilot reads altitude, heading, and speed from three different instruments at a glance.

The tool for counting independent channels is principal component analysis. Apply it to the rows of the transition matrix, and the number of significant components is the **effective dimensionality** d_{eff} —the number of questions the syntax answers for free. The degree to which no single component dominates is **integration**, written Φ [3]. High Φ means many independent channels. $\Phi = 0$ means the syntax is a one-note instrument: every line has the same shape, and you must read the words to know what they do.

3.2 Why these quantities

These three quantities are not arbitrary. Each connects to a foundational theory.

Integration (Φ) comes from Integrated Information Theory, proposed by neuroscientist Giulio Tononi in 2004 to formalize consciousness [3]. A system with high Φ is one whose parts interact in ways that cannot be decomposed—the whole is more than the sum of its parts. Applied to syntax: a language with high integration provides multiple independent channels of

orientation. A language with $\Phi = 0$ is a pipeline—every token follows every other in the same pattern, and the syntax provides no structural variety.

Optionality (S_τ) comes from the theory of causal entropic forces, published by Wissner-Gross and Freer in 2013 [2]. They showed that systems which maximize the diversity of their accessible futures spontaneously exhibit intelligent behavior. Applied to syntax: a language with high optionality gives the programmer many meaningfully different continuations at each position. Low optionality channels everyone into the same patterns.

Together: high optionality means many choices. High clarity means you can tell them apart. High integration means the information arrives on independent channels. A language that maximizes all three is one where the programmer has many options, can distinguish them at a glance, and processes the orientation on parallel cognitive channels.

3.3 The results

We computed all three for eight languages. Look at the last column.

Table 2: Syntactic metrics for existing languages ($\tau = 3$). No existing language exceeds $d_{\text{eff}} = 2$.

Language	Optionality (S_τ)	Clarity (JSD)	Integration (Φ)	d_{eff}
Prolog	2.891	0.688	0.937	2
Haskell	3.012	0.671	0.883	2
Scala	3.115	0.589	0.412	1
Python	2.743	0.749	0.621	1
Rust	2.987	0.634	0.000	1
Kotlin	3.045	0.612	0.000	1
Lisp	2.456	0.523	0.312	1
C	2.834	0.601	0.189	1

Rust has integration $\Phi = 0.000$. The most carefully designed systems language in mainstream use answers *zero* questions for free. Every line starts with a keyword that flows into an identifier that flows into braces—the same visual shape whether you are defining a function, a struct, a module, or an if-block. The syntax tells you nothing until you read the keyword. Every question requires decoding.

Rust programmers know this feeling. You open a 500-line file and scroll through blocks that all *look the same* until you slow down and read the first word of each line. That is what $\Phi = 0$ feels like.

The highest-scoring language is Prolog—the only language in the set where clause heads flow differently from clause bodies, which flow differently from queries. Two genuinely different pathways. Two dimensions.

But nobody reaches three.

4 The Bottleneck

Why? The answer is a structural pattern shared by every multi-paradigm language.

In Rust, the keywords `fn`, `struct`, `impl`, `let`, and `if` are *semantically* different but *syntactically* identical. They all flow into an identifier followed by braces. The transition matrix cannot tell them apart, so the independent dimensions collapse into one.

In Kotlin and Scala, every token category connects to every other with roughly equal probability. Maximum syntactic flexibility produces maximum cognitive uniformity.

The key insight: the constraint that creates cognitive structure is not richness. It is *differentiation*. Not how many things you can say, but how sharply the syntax distinguishes between the *kinds* of things you are saying.

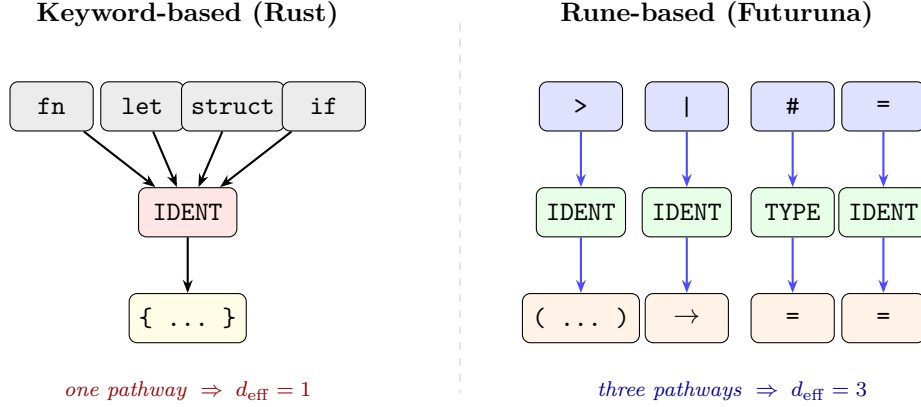


Figure 1: **The bottleneck.** Keywords (left) funnel all statements through $\text{IDENT} \rightarrow \{ \dots \}$ —one pathway, one dimension. Runes (right) create distinct chains ($> \rightarrow \text{IDENT} \rightarrow (\dots)$, $| \rightarrow \text{IDENT} \rightarrow \rightarrow$, $\# \rightarrow \text{TYPE} \rightarrow =$). Three independent pathways, three principal components, three cognitive dimensions.

We ran NSGA-II [7], an evolutionary search that optimizes multiple goals at once, over the space of possible 22×22 transition matrices. The three goals: maximize optionality, clarity, and integration simultaneously. Population 500, 200 generations. The result was a Pareto frontier: the set of 122 designs where you cannot improve any one metric without worsening another. The best possible trade-offs.

85 of them achieve $d_{\text{eff}} \geq 3$. All 85 share one structural feature: a strong $\text{START} \rightarrow \text{OP}$ transition. Statements begin with an operator, not a keyword.

Claim 1. *Statement runes are necessary and sufficient for $d_{\text{eff}} = 3$ in the measured token transition framework.*

No frontier member achieves $d_{\text{eff}} = 3$ without them. Every member with them achieves $d_{\text{eff}} \geq 3$.

Principal component analysis on the rune-based transition matrix finds three independent dimensions (λ : 2.28, 1.66, 0.96):

Axis 1 — Statement kind ($\lambda = 2.28$). The rune at the start of each line. Your first cognitive act: *what kind of statement is this?*

Axis 2 — Type flow ($\lambda = 1.66$). Type signatures ($\text{Int} \rightarrow \text{Bool} \rightarrow \text{String}$) flow independently of statement kind. This axis exists in Haskell. In rune-based syntax, it decouples from Axis 1 because runes, not keywords, carry the statement’s nature.

Axis 3 — Block composition ($\lambda = 0.96$). How braces nest. In C-family languages this axis is entangled with Axis 1—keywords like **fn** and **class** initiate both statement kind *and* block structure. Runes separate them.

Keywords overload two cognitive functions: they signal statement kind *and* initiate blocks. The rune decouples these. The decoupling creates independence. The independence creates the third dimension.

5 Seven Runes

If runes unlock the third dimension, how many? The Pareto frontier constrains the answer: seven to nine produce the optimal balance of optionality, clarity, and integration. Seven is the

sweet spot. They were not chosen by taste. They fell out of a question: *what are the kinds of things a program can say?*

A program describes structures that can exist, processes that can happen, truths that must hold, facts that are observed, flows that evolve in time, boundaries where formal reasoning ends, and demands for verification.

Seven categories. Each gets a rune.

Table 3: The seven runes of Futuruna. Each answers one epistemic question.

Rune	Question	Verification target	Examples
#	What exists?	Z3 datatypes	Types, effects, traits
>	What happens?	Z3 functions	Functions, actors, modules
	What must be true?	Z3 assertions	Rules, match arms, handlers
=	What is?	Z3 constants	Bindings, ground truth
~	What flows?	Temporal logic	Reactive streams, time
@	Where do proofs stop?	Proof boundary	IO, imports, meta
?	Prove it.	Solver invocation	Verification demands

We conjecture this set is closed. Every proposed eighth rune we have tested reduces to one of these seven. Permissions are constraints on structures: #. Concurrency is processes that happen in parallel: >. Error handling is case analysis: |. Macros are boundary operations: @. The seven-fold partition mirrors a natural decomposition of what programs express—ontology, dynamics, logic, observation, time, boundary, proof. We have not found a program statement that requires a genuinely new direction. Whether this partition is provably complete remains an open question; what we can show is that it covers every statement kind in the eight measured languages without overlap.

5.1 # — What exists

Types, algebraic effects, traits, implementations. In other languages, these are scattered across `class`, `struct`, `enum`, `interface`, `trait`, and `type`. In Futuruna, one rune:

```
# List(T) = Nil | Cons(head: T, tail: List(T))
# Weather(temp: Float, condition: Condition, wind_kph: Float)

# effect Console {
  > say(msg: String) → ()
  > ask(prompt: String) → String
}

# trait Printable { > display(self) → String }
```

5.2 > — What happens

Functions, actors, modules. Everything that describes transformation:

```
> add(a: Int, b: Int) → Int { a + b }

> actor counter(state: Int) {
  | Increment → state + 1
  | Decrement → state - 1
}
```

5.3 = — What is

Bindings. Ground truth. A point in time:

```
= x = 42
= name = "hello"
= result <- parse_int("42")    -- monadic bind (unwrap or early-return)
```

5.4 ~ — What flows

Where = binds a point in time, ~ binds a line through time:

```
~ clicks = events("button", "click")
~ count = clicks |> s_scan(0, |n, _| n + 1)
~ label = count |> s_map(|n| "Count: " + show(n))
```

In every other language, reactive streams are a library—RxJS, Reactor, Akka Streams. In Futuruna, they are syntax. The compiler sees the stream topology at compile time.

5.5 @ — Where proofs stop

The boundary between the verified world and the effectful world. Every use of @ says: *here, formal reasoning cannot reach.*

```
@ print("hello")                -- IO
@ use std::collections::HashMap -- external code
@ depend "serde" "1"            -- build system
@ comptime = table = gen(1000)   -- compile-time evaluation
@ rust { fn fast(x: &[f64]) { x.sort_unstable(); } }
```

5.6 ? — Prove it

At first glance, ? seems simple: check a predicate, stop. The full picture is richer. Every other rune declares something. ? interrogates it. The predicate comes from > and |. The subject comes from ~ and =. The reaction can use @. At each call site, the programmer decides: is violation fatal, recoverable, or informational?

Six forms, from simplest to most expressive. The key distinction: without an **else** block, failure halts the program. With an **else** block, failure is handled and execution continues.

```
-- Bare check: halt on failure, continue on success
? balance_bounded

-- Pass block: run code on success, halt on failure
? balance_bounded → { @ print("Balance OK") }

-- Capture + pass: bind the checked value, halt on failure
? balance_bounded: val → { @ print("Value: " + show(val)) }

-- Else block: handle failure, no halt
? balance_bounded else { @ print("VIOLATION") }

-- Pass + else: handle both outcomes, no halt
? balance_bounded → { @ print("OK") } else { @ print("FAIL") }

-- Full form: capture + pass + else, no halt
? balance_bounded: val → {
```

```

    @ print("Verified: " + show(val))
} else {
    @ print("Violation detected")
}

```

You define what “correct” means declaratively, using `|` rules. Then at each call site you choose how to react. Without `else`, `?` is a hard assertion—failure stops the program. Add an `else` block and it becomes a recovery point—failure is handled, execution continues. The `:` `val` capture binds the subject value (the data being checked, not the boolean result), feeding verified data downstream. This is verification and error handling unified under one character.

The three assurance levels from the earlier section still apply to every form: `runa run` evaluates the predicate. `runa build` emits `debug_assert!()`. `runa verify` invokes Z3 for a proof over all inputs. The shape of the reaction is independent of the depth of the proof.

6 A Complete Program

All seven runes in one file. Read the first character of each line as an epistemic declaration.

```

# Phase = Idle | Active | Done

> add(a: Int, b: Int) → Int { a + b }
> clamp(val: Int, lo: Int, hi: Int) → Int {
    if val < lo { lo }
    else { if val > hi { hi } else { val } }
}

= x = 42
= y = 10
= balance = 1000
= max_supply = 1000000

| sum_positive: add(x, y) → add(x, y) > 0
| balance_bounded: balance → balance >= 0 && balance <= max_supply
| commutative: add(x, y) → add(x, y) == add(y, x)

~ ticks = from_list([1, 2, 3, 4, 5])
~ evens = ticks |> s_filter(|n| n % 2 == 0)

@ print("--- Verifying ---")

? sum_positive
? balance_bounded: val → { @ print("Balance: " + show(val)) }
? commutative else { @ print("Commutativity violated!") }
? all

```

Interpreter output:

```

--- Verifying ---
[ok] |sum_positive| holds (value: 52)
[ok] |balance_bounded| holds - Balance: 1000
[ok] |commutative| holds (value: 52)
[ok] all 3 invariant(s) hold

```

Three different `?` forms in one program: a bare assertion, a value capture with a pass block, and an assertion with a recovery branch. With `runa verify`, all three invoke Z3 to prove each invariant for *all* inputs. The reaction shape is orthogonal to the proof depth.

7 Measured Comparison

The number to watch is the last column.

Table 4: Futuruna vs. existing languages. No existing language reaches $d_{\text{eff}} = 3$.

Design	Optionality	Clarity	Integration	d_{eff}	Note
Prolog	2.891	0.688	0.937	2	Most integrated among existing
Haskell	3.012	0.671	0.883	2	Best composite among existing
Scala	3.115	0.589	0.412	1	Most options among existing
Python	2.743	0.749	0.621	1	Clearest among existing
Rust	2.987	0.634	0.000	1	Zero integration
Futuruna	3.537	0.784	0.980	3	Statement runes [†]

[†]Futuruna metrics computed from the language’s 30+ test programs. Production-scale codebases—where function bodies and bindings dominate—may produce different numbers. The structural claim ($d_{\text{eff}} = 3$) rests on the rune mechanism, not the specific metric values.

Futuruna dominates every measured language on all three axes simultaneously. More options than Scala. Clearer than Python. More integrated than Prolog. And the only design that reaches the third dimension.

What does this feel like? A Futuruna programmer scanning unfamiliar code has three questions answered before processing a single identifier. The rune says what *kind* of statement. The type signatures say what *types* flow through it. The block structure says where it *sits*. Three facts, none redundant with the others, all arriving through separate visual cues that the brain processes in parallel.

Consider traffic signs. Imagine a city with no signs at all—just white text painted on the road surface. “STOP.” “YIELD.” “SPEED LIMIT 50.” Every instruction looks the same until you read the words. That is $\Phi = 0$: zero channels of pre-attentive information. You must decode every message. Now mount them on poles as round white signs with black text. You know something is a sign before you read it—that is $d_{\text{eff}} = 1$: one channel (sign versus not-sign). Now add color: red means prohibition, blue means information, yellow means caution. You react to the color before you read the word. That is $d_{\text{eff}} = 2$: two channels. Now add shape: octagons for stop, triangles for yield, rectangles for information. Shape, color, and text arrive on three parallel channels. You know what *kind* of sign it is, what *category* of instruction it carries, and what the *specific* message says—all before conscious reading begins. That is $d_{\text{eff}} = 3$.

Traffic signs work this way because the human visual system processes shape, color, and symbol on independent channels. Programming language syntax faces the same constraint: a programmer scanning unfamiliar code is a driver entering an unfamiliar intersection. Rust is the white text on asphalt—every line looks the same until you read the keyword. Futuruna is the shaped, colored sign: the rune tells you the kind, the type signature tells you the flow, and the block depth tells you the structure. Three questions, answered before conscious decoding begins.

8 The Formal Definitions

For completeness, the formal definitions behind the three named quantities.

Definition 1 (Optionality S_τ). For transition matrix P with $(P)_{ij} = w(i, j) / \sum_k w(i, k)$:

$$S_\tau = \sum_i \pi_i \cdot H(P^\tau \cdot \delta_i), \quad \text{where} \quad H(\mathbf{p}) = - \sum_j p_j \log_2 p_j$$

π_i is the stationary weight of category i , δ_i is the unit vector at node i , $\tau = 3$.

Definition 2 (Clarity (JSD)).

$$\text{JSD} = \frac{1}{\binom{|V|}{2}} \sum_{i < j} \left[\frac{1}{2} D_{\text{KL}}(P_i \| M_{ij}) + \frac{1}{2} D_{\text{KL}}(P_j \| M_{ij}) \right], \quad M_{ij} = \frac{P_i + P_j}{2}$$

Definition 3 (Integration Φ and dimensionality d_{eff}). Let C be the covariance matrix of the rows of P , with principal component variances $\lambda_1 \geq \lambda_2 \geq \dots$. Then:

$$d_{\text{eff}} = |\{i : \lambda_i > \epsilon\}| \quad \Phi = 1 - \frac{\lambda_1}{\sum_i \lambda_i}$$

9 Implementation

Futuruna is a 10,000+ line Rust compiler (**runa.rs**) with four modes:

runa run file.runa Interpret directly. `?` performs runtime checks.
runa emit file.runa Transpile to Rust, display generated code.
runa build file.runa Transpile to Rust, compile to native binary.
runa verify file.runa Invoke Z3 for formal proofs of all invariants.

9.1 Ownership without annotation

Futuruna is to Rust as Kotlin is to Java. The programmer writes value semantics; the compiler handles ownership. Whole-program escape analysis decides: single-use variables move, multi-use variables clone, read-only parameters auto-borrow. The programmer never writes `&T`, never annotates a lifetime, never calls `.clone()`. An escape hatch (`@ rust {}`) covers the remaining 5%.

9.2 The verification pipeline

The runes are not decoration. They are the interface to the verifier. Each maps directly to a category in the standard theorem-prover input format (SMT-LIB2):

- `# algebraic types` → `declare-datatypes`
- `> functions` → `define-fun`
- `= bindings` → `define-const`
- `| invariants` → `assert` $\neg$ predicate (refutation)

The seven runes partition a program into exactly the categories a formal methods tool needs. The syntax *is* the verification interface.

10 Honest Limitations

- **The metrics are proxies.** Optionality, clarity, and integration measure syntactic texture, not programmer productivity. Three cognitive axes *should* improve the experience of reading code, but this has not been tested in user studies. The metrics are a map, not the territory.
- **Corpus dependence.** Transition matrices depend on which code samples represent each language. Systems code and web code produce different numbers. Representativeness is inherently debatable.
- **Verification scope.** The `-verify` pipeline handles integer arithmetic, algebraic data types, and first-order functions. Higher-order functions and floating-point require extensions not yet shipped.

- **Reactive streams.** The `~` rune supports both synchronous **Vec**-based and asynchronous multi-threaded semantics. Timing operators (debounce, throttle) are not yet implemented.
- **Maturity.** Futuruna is a working compiler with 30+ test programs, not a production language. The escape hatch covers the gaps.

The deepest limitation is the first. We measured the information-theoretic structure of token transition graphs and found something real. Whether the third cognitive dimension translates to the lived experience of writing code is an empirical question this paper opens but does not close.

The strongest version of this work’s contribution is not “Futuruna is better” but “measurement reveals structure that intuition missed.” If someone builds a better language using different measurements, the methodology wins.

11 Related Work

Integrated Information Theory [3] provides the framework for integration (Φ). Tononi developed IIT to formalize consciousness in neural systems; we apply it to the covariance structure of token transitions. **Causal entropic forces** [2] provide optionality (S_τ); Wissner-Gross and Freer showed that maximizing path entropy produces intelligent behavior in physical systems; we show it characterizes syntactic freedom.

Catala [4] pioneered legal default logic; Futuruna absorbs this into `|`. **Koka** [5] proved algebraic effects are tractable; Futuruna adopts them via `# effect` and `| handle`. **Unison** [6] showed content-addressed code eliminates dependency hell; Futuruna implements it via AST hashing. **Zig** demonstrated compile-time evaluation eliminates macros; Futuruna adopts `@ comptime`. **Hylo** explored mutable value semantics; Futuruna adopts `inout`.

No prior language has combined these features, and none has derived its syntax from quantitative measurement.

12 Conclusion

We measured the cognitive structure of programming language syntax and found that every existing language occupies at most two dimensions. A third dimension is available. It requires a constraint that is not the one intuition suggests.

Not richness. Not flexibility. Not multi-paradigm abundance.

Differentiation.

One character at the start of each line that says: *this is what kind of thing I am about to tell you*. Seven characters, covering every category of program statement we have found. Each one maps to a verification target. Each one creates a distinct cognitive pathway.

The language that results unifies things that have never coexisted: logic programming and algebraic effects and legal default logic and reactive streams, all under a syntax that sits on the Pareto frontier for the information-theoretic proxies we have. You cannot improve one metric without worsening another. It compiles to native Rust with inferred ownership. It provides formal verification through the same runes that structure the code. And it occupies a region of the design space that no existing language has reached.

Remember the feeling from the first paragraph. You open a file and within seconds you know whether you are in friendly territory. That feeling has a name now. It is d_{eff} —the number of questions the syntax answers before you read a single word.

Every existing language leaves at least one of those questions unanswered.

References

- [1] C. E. Shannon. A mathematical theory of communication. *Bell System Technical Journal*, 27(3):379–423, 1948.
- [2] A. D. Wissner-Gross and C. E. Freer. Causal entropic forces. *Physical Review Letters*, 110:168702, 2013.
- [3] G. Tononi. An information integration theory of consciousness. *BMC Neuroscience*, 5:42, 2004.
- [4] D. Mériçoux, N. Chataing, and J. Protzenko. Catala: A programming language for the law. *Proc. ACM on Programming Languages*, 5(ICFP):1–29, 2021.
- [5] D. Leijen. Type directed compilation of row-typed algebraic effects. In *Proc. POPL*, pages 486–499, 2017.
- [6] P. Chiusano and R. Bjarnason. Unison: a new approach to distributed programming. <https://www.unison-lang.org/>, 2019.
- [7] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan. A fast and elitist multiobjective genetic algorithm: NSGA-II. *IEEE Trans. Evolutionary Computation*, 6(2):182–197, 2002.
- [8] L. de Moura and N. Bjørner. Z3: An efficient SMT solver. In *Proc. TACAS*, pages 337–340, 2008.